

# Operating Systems

Rowan University

# Overview

1. Process & Thread Review
2. Interprocess Communication & Synchronization
3. Race Conditions
4. Semaphores

# Process States for Xinu

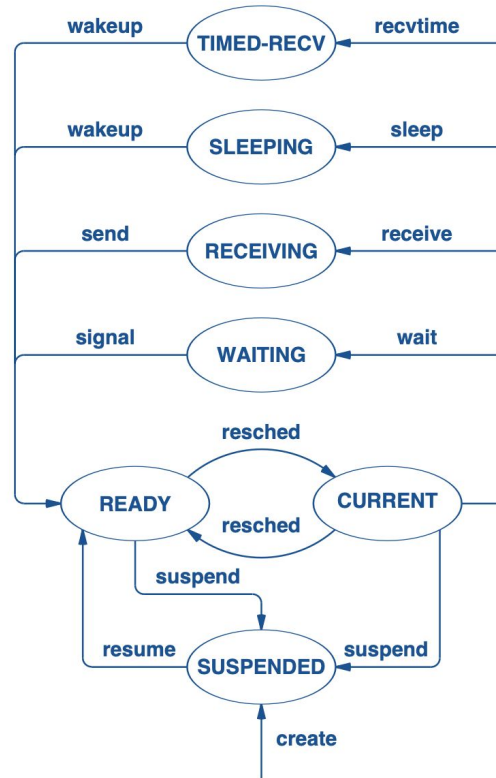


Figure 13.3 State transitions including the *timed receive* state.

# Example: Shell Forks Process

Figure 10-8

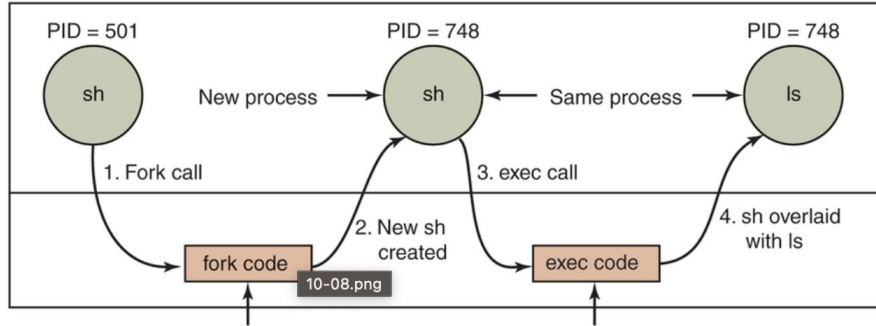


Figure 1-19

```
#define TRUE 1

while (TRUE) {
    type_prompt( );
    read_command(command, parameters);

    if (fork() != 0) {
        /* Parent code. */
        waitpid(-1, &status, 0);
    } else {
        /* Child code. */
        execve(command, parameters, 0);
    }
}
```

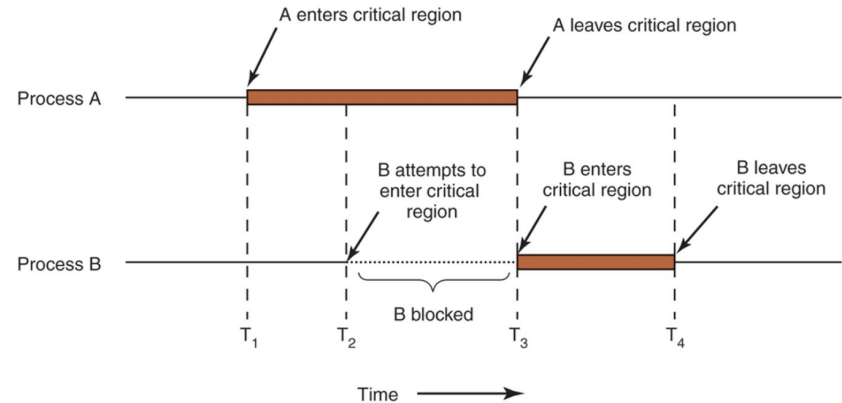
/\* repeat forever \*/  
/\* display prompt on the screen \*/  
/\* read input from terminal \*/  
  
/\* fork off child process \*/  
  
/\* wait for child to exit \*/  
  
/\* execute command \*/

A stripped-down shell. Throughout this book, *TRUE* is assumed to be defined as 1.

# Critical Region

- Implement mutual exclusion to prohibit simultaneous read and write operations
  - If one process accesses shared data others are excluded from access
- Part of the program where shared memory is accessed is called the *critical region*
  - No two processes are simultaneously inside their critical regions

Figure 2-22



Mutual exclusion using critical regions.

# Producer Consumer Model

- Common for Distributed Systems (Apache Kafka)
- Two processes share a common, fixed-size buffer
  - Can be scaled to many processes
  - Producer will sleep if the buffer is full
- Part of Sleep & Wakeup approach to IPC
- **Uses a bounded-buffer**

Figure 2-27

```
#define N 100
int count = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N) sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1) wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0) sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1) wakeup(producer);
        consume_item(item);
    }
}
```

*/\* number of slots in the buffer \*/*  
*/\* number of items in the buffer \*/*

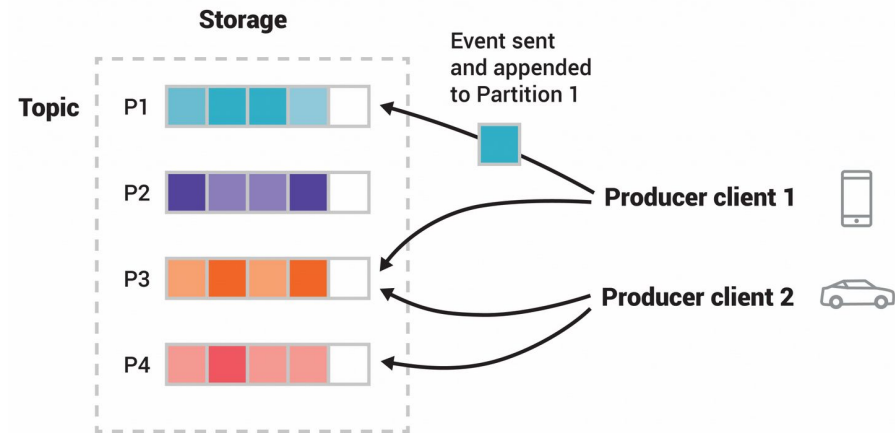
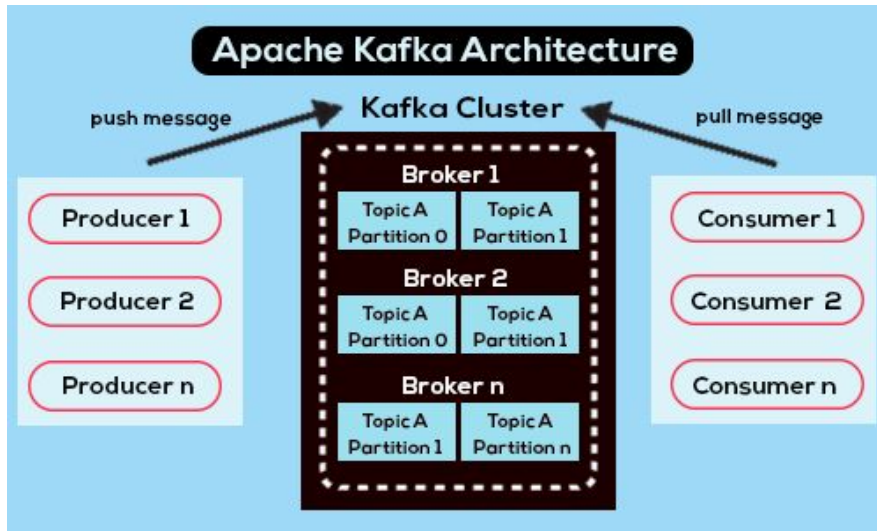
*/\* repeat forever \*/*  
*/\* generate next item \*/*  
*/\* if buffer is full, go to sleep \*/*  
*/\* put item in buffer \*/*  
*/\* increment count of items in buffer \*/*  
*/\* was buffer empty? \*/*

*/\* repeat forever \*/*  
*/\* if buffer is empty, got to sleep \*/*  
*/\* take item from buffer \*/*  
*/\* decrement count of items in buffer \*/*  
*/\* was buffer full? \*/*  
*/\* print item \*/*

The producer-consumer problem with a fatal race condition.

# Apache Kafka

[https://sites.pitt.edu/~viz/classes/infsci3350/resources/linkedin\\_icde12.pdf](https://sites.pitt.edu/~viz/classes/infsci3350/resources/linkedin_icde12.pdf)



# Semaphores

- Similar to a binary mutex lock except it acts as a queue of waiting processes
- Semaphore is an integer
  - Equal to the number of process waiting to be woken up
- Process sleeps when semaphore is 0 and `down` is called on it
  - `down(&full)`

Figure 2-28

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

```
/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */
```

```
/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */
```

```
/* infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */
```

# Reader Writer Problem

- Example:
  - Modeling access to a database in scenarios with multiple processes
  - An airline reservation system with concurrent read and write operations
- Solution
  - Readers: Use a semaphore (`db`) to control access. Increment a counter (`rc`) for readers. Manage entry and exit from the critical section.
  - Writers: Employ the semaphore (`db`) for exclusive access during write operations.
- A writer may be delayed indefinitely if a steady stream of readers is present (DoS)

Figure 2-29

```
typedef int semaphore;
semaphore mutex = 1;
semaphore db = 1;
int rc = 0;

void reader(void)
{
    while (TRUE) {
        down(&mutex);
        rc = rc + 1;
        if (rc == 1) down(&db);
        up(&mutex);
        read_data_base();
        down(&mutex);
        rc = rc - 1;
        if (rc == 0) up(&db);
        up(&mutex);
        use_data_read();
    }
}

void writer(void)
{
    while (TRUE) {
        think_up_data();
        down(&db);
        write_data_base();
        up(&db);
    }
}
```

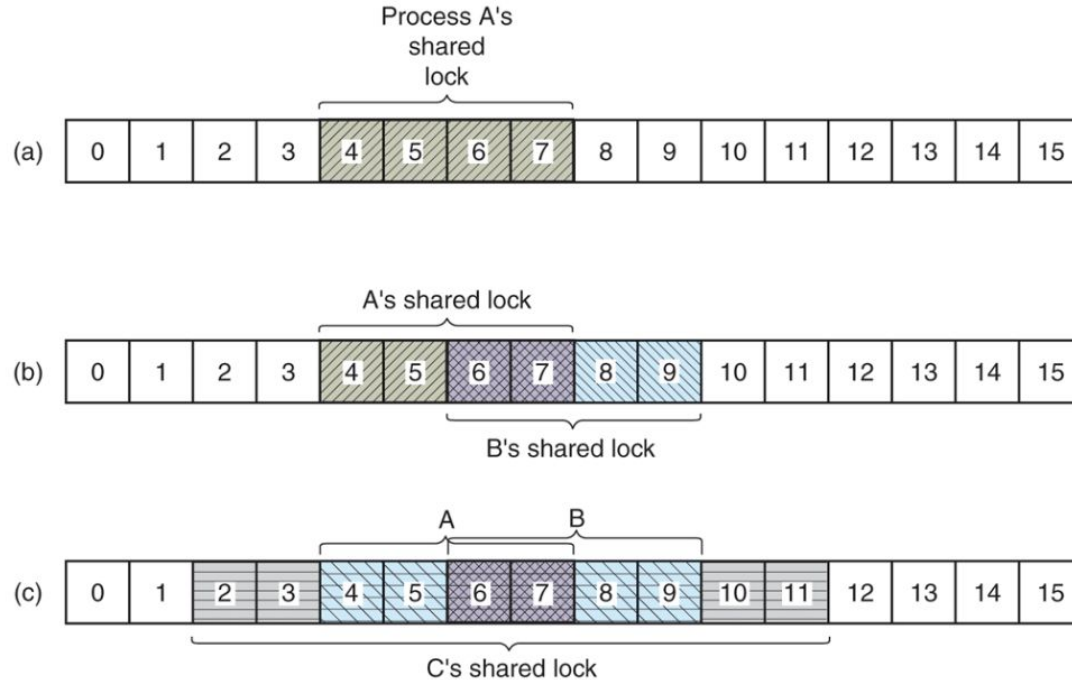
```
/* use your imagination */
/* controls access to rc */
/* controls access to the database */
/* # of processes reading or wanting to */

/* repeat forever */
/* get exclusive access to rc */
/* one reader more now */
/* if this is the first reader ... */
/* release exclusive access to rc */
/* access the data */
/* get exclusive access to rc */
/* one reader fewer now */
/* if this is the last reader ... */
/* release exclusive access to rc */
/* noncritical region */

/* repeat forever */
/* noncritical region */
/* get exclusive access */
/* update the data */
/* release exclusive access */
```

# Linux File Locking

Figure 10-26



(a) A file with one lock. (b) Adding a second lock. (c) A third one.

# Mutex Revisited

- Mutex (Mutual Exclusion) simplifies semaphore for exclusive access in shared resources or code sections
  - Especially useful in user-space thread packages
- Shared variable with two states: Unlocked or Locked
  - Typically represented as an integer, any value not 0 is considered locked

**Figure 2-30**

|                    |                                                 |
|--------------------|-------------------------------------------------|
| mutex_lock:        |                                                 |
| TSL REGISTER,MUTEX | I copy mutex to register and set mutex to 1     |
| CMP REGISTER,#0    | I was mutex zero?                               |
| JZE ok             | I if it was zero, mutex was unlocked, so return |
| CALL thread_yield  | I mutex is busy; schedule another thread        |
| JMP mutex_lock     | I try again                                     |
| ok:                | RET                                             |
|                    | I return to caller; critical region entered     |
| mutex_unlock:      |                                                 |
| MOVE MUTEX,#0      | I store a 0 in mutex                            |
| RET                | I return to caller                              |

Implementation of *mutex\_lock* and *mutex\_unlock*.

# Considerations

- User threads run without interruption from the OS
  - OS interrupts the process not a thread
- Mutex in User Space
  - When the mutex is locked, thread can give up execution with `thread_yield`
  - Allow other threads to run and check back later
- Mutexes existed in shared address space of threads

Figure 2-25

```
enter_region:
    TSL REGISTER,LOCK
    CMP REGISTER,#0
    JNE enter_region
    RET
```

I copy lock to register and set lock to 1  
I was lock zero?  
I if it was not zero, lock was set, so loop  
I return to caller; critical region entered

```
leave_region:
    MOVE LOCK,#0
    RET
```

I store a 0 in lock  
I return to caller

Entering and leaving a critical region using the TSL instruction.

Figure 2-30

```
mutex_lock:
    TSL REGISTER,MUTEX
    CMP REGISTER,#0
    JZE ok
    CALL thread_yield
    JMP mutex_lock
ok:      RET
```

I copy mutex to register and set mutex to 1  
I was mutex zero?  
I if it was zero, mutex was unlocked, so return  
I mutex is busy; schedule another thread  
I try again  
I return to caller; critical region entered

```
mutex_unlock:
    MOVE MUTEX,#0
    RET
```

I store a 0 in mutex  
I return to caller

Implementation of *mutex\_lock* and *mutex\_unlock*.

# Calls to Create Mutex

Figure 2-31

| Thread call                        | Description               |
|------------------------------------|---------------------------|
| <code>pthread_mutex_init</code>    | Create a mutex            |
| <code>pthread_mutex_destroy</code> | Destroy an existing mutex |
| <code>pthread_mutex_lock</code>    | Acquire a lock or block   |
| <code>pthread_mutex_trylock</code> | Acquire a lock or fail    |
| <code>pthread_mutex_unlock</code>  | Release a lock            |

Some of the Pthreads' calls relating to mutexes.

# Conditional Variables

- Used for threads to block on conditions not being met
- Mutexes and Condition Variables Together:
  - Always used together in synchronization
  - One thread locks a mutex, then waits on a condition if necessary
  - Another thread signals or broadcasts to wake up waiting threads
- Signal Loss:
  - Condition variables have no memory
  - If a signal is sent when no thread is waiting, the signal is lost
  - Programmers must ensure signals are not lost

Figure 2-32

| Thread call            | Description                                  |
|------------------------|----------------------------------------------|
| pthread_cond_init      | Create a condition variable                  |
| pthread_cond_destroy   | Destroy a condition variable                 |
| pthread_cond_wait      | Block waiting for a signal                   |
| pthread_cond_signal    | Signal another thread and wake it up         |
| pthread_cond_broadcast | Signal multiple threads and wake all of them |

Some of the Pthreads calls relating to condition variables.

# Process States for Xinu

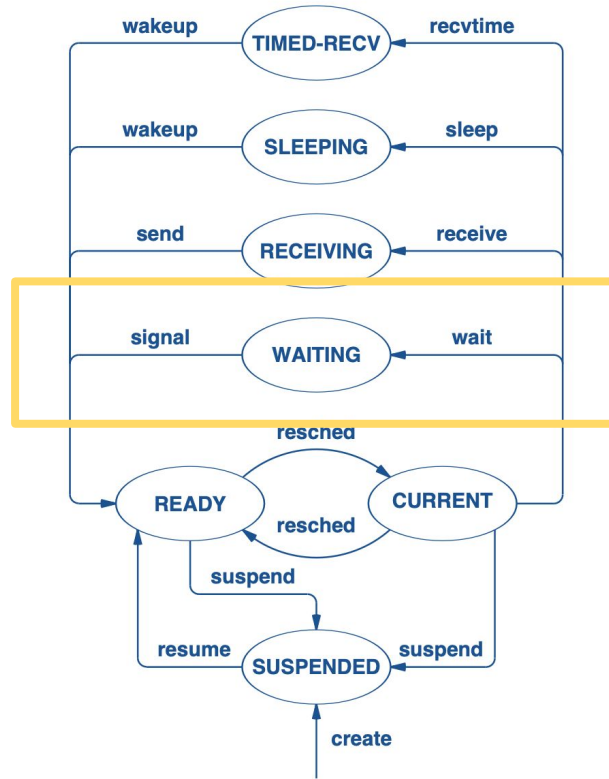


Figure 13.3 State transitions including the *timed receive* state.

# Monitors

- Higher-level synchronization primitive
- A collection of procedures, variables, and data structures grouped together
- Monitor Operations:
  - *Only one process active in a monitor at any instant*
  - Compiler handles monitor procedure calls differently
  - Automatic mutual exclusion prevents race conditions

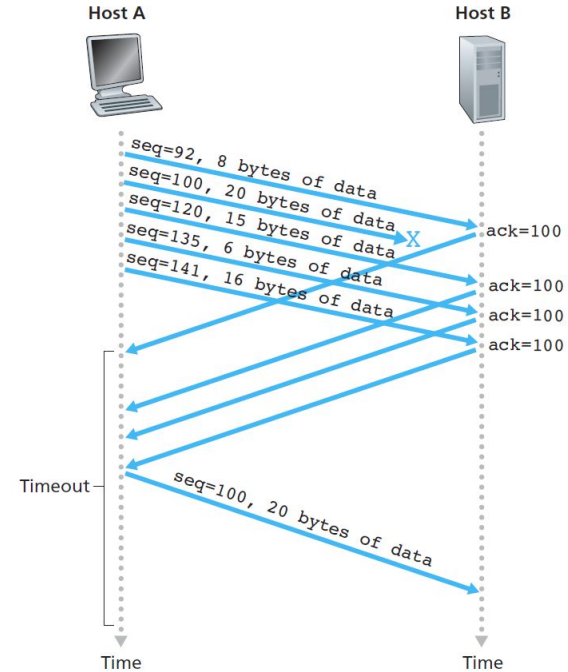
```
monitor example
    integer i;
    condition c;

    procedure producer( );
    .
    .
    end;

    procedure consumer( );
    .
    .
    end;
end monitor;
```

# Message Passing

- Message Passing: Interprocess communication using system calls
- Primitives: `send` and `receive`
- Design Issues in Message-Passing Systems:
  - Challenges in distributed systems
  - Lost messages: Agreement on acknowledgment
  - Handling message retransmission with sequence numbers
  - Naming processes and authentication



# Xinu Galileo

- Examine the Process implementation in Xinu
- Link to source code in Github has been added to Canvas
- <https://github.com/real-xinu/xinu-galileo/blob/master/include/process.h>
- <https://github.com/real-xinu/xinu-galileo/blob/master/include/kernel.h>